Misinformation Detection in News Headlines

COMP6713 - Natural Language Processing

News Group

Lachlan Johnston (z5477322)

Sebastian Litchfield (z5477323)

Jaiden Brocklebank (z5421748)

PART A: Problem Definition			
Minimum: 10 credits		*Fill your project details here*	
Criterion	**Points**	**Details**	**Your Response**
NLP Problem	5 credits per problem	Examples: Question-answering, Sentiment Analysis, etc. (Refer to course modules)	1. Text Classification (misinformation detection) 2. Information Retrieval (fact checking with LangChain)
Text Source/Domain	5 credits per text source/domain	Examples: News Articles, Medical papers, Social media posts, etc.	1. News articles / headlines
PART B: Dataset Selection			
Minimum: 20 credits			
Criterion	**Points**	**Details**	**Your Response**
Use two existing datasets	10 credits	Publicly available datasets.	1. LIAR dataset: https://huggingface.co/datasets/ucsbnlp/liar 2. FakeNewsNet dataset: https://github.com/KaiDMML/FakeNewsNet

Create your own labelled dataset	20 credits	Correct selection of labels, inter-annotator agreement	
Use an existing lexicon	10 credits	Examples: WordNet, medical ontology	1. WordNet
PART C: Modelling			
Minimum: 30 credits			
Criterion	**Points**	**Details**	**Your Response**
Implement a rule-based or statistical model as a baseline	*Essential*	Please keep this approach simple – this is only a baseline.	TF-IDF + Logistic Regression
Use an existing pre-trained/ fine-tuned model	5 credits per model	You must compare the performance of multiple models to accrue credits, if you are only using available fine-tuned models or only prompting.	Compare BERT to fine tuned BERT to RoBERTa
Fine-tune a model based on a dataset	20 credits per out-of-the-box fine-tuning method	Examples: fine-tuned BERT, prefix-tuned LLaMA	Fine tuning BERT on LIAR dataset
Extend a method	30 credits per extended fine-tuning method	Examples: modification of the loss function, incorporation of structured ontology, prompting method other than zero/few-shot prompting, etc.	
Integrate a language model with external tools	20 credits	Usage of a library like LangChain/LangSmith	Use LangChain to fact check from external sources. This contributes to the output of the model by

			adding weight to the confidence.
PART D: Evaluation			
Minimum: 20 credits			
Criterion	**Points**	**Details**	**Your Response**
Quantitative Evaluation	10 credits	Appropriate metrics.	F1 Score, Precision, Recall, Accuracy.
Qualitative Evaluation	5 credits	Examine mis-classified instances and produce common error types	Perform a manual analysis of misclassified statements, looking for any patterns
Command line testing	5 credits	Interface to test out the system. This can be executed using an input argument or input file.	Simple CLI interface where you put in a news article headline and it will return a classification.
Demo	10 credits	A simple demo through Gradio (or equivalent).	Simple Gradio demo for interactive testing of the final system.

1. Introduction

The modern consumption of digital media and news has significantly shifted the landscape of public information. Whilst digital access has improved access to democratised news, it also creates an environment for the rapid creation of misinformation. The speed and scale at which information can be shared outpaces previous forms of traditional media, allowing for false or misleading narratives to gain traction before they can be effectively verified. Much of the “fake news” seen today is complex, and often involves sophisticated linguistic techniques to bypass initial skepticism of readers. As misinformation evolves, it becomes ever more important to implement multi-modal detection systems.

We aim to address these challenges by developing comprehensive, multi-layered text classification models and information retrieval systems that are capable of identifying if an article headline is real. The testing and development process will be recorded, and findings on various methodologies and techniques will be noted in an attempt to optimize the model and overall system. The overarching goal is to use transformer-based models (BERT and RoBERTa), and enhance their capabilities alongside Wordnet and external fact-checking via

LangChain tools such as Google's Serper, and Gemini. By doing so, we are able to produce a system that not only verifies headlines, but validates sources in real-time.

2. Problem Definition

2.1 NLP Problems

In this project we address two primary Natural Language Processing (NLP) problems, that being text classification and information retrieval. The problem and goal of text classification is to categorize a headline (input string) into a predefined set of labels, for example true, false, or mixed depending on the contents of the input. We use transformer-based architectures BERT and RoBERTa, to perform deep training and analysis select datasets as a way of identifying fake news solely based on the language used. The limitation of this method stems from the model's inability to adapt / receive new forms of information, as it is limited to its pre-trained knowledge.

Information retrieval or external fact-checking tools aim to overcome this constraint, by expanding the model's ability and allowing it to fact-check in real time. This involved additional tooling through LangChain, enhancing the model with Google search functionalities to verify the inputted headline against real world events and articles. Gemini is used to automate this process, allowing it to autonomously evaluate the results and output a final verdict. A problem with this strategy is that gathered information is subject to misinterpretation by Gemini. To combat this, we fine-tune Gemini's output by tweaking prompts, weighing importance on reputable sources retrieved, and adjusting metrics to reduce drastic / misinformed decisions by Gemini.

2.2 Text Domain

For this project the text domain is specifically focused on news headlines and political statements. We focused on this area as we find identifying fake news in the political space is especially important. Misinformation in this space is often professionally created to mimic the structural and stylistic tendencies of modern journalism. These fabricated headlines look and sound professional in nature, making them increasingly difficult to detect whilst fooling more people with ease. By narrowing down the text domain to headlines and short-form statements, we aim to target the growing issue of misinformation in relation to news and political headlines.

3. Dataset Selection

3.1 LIAR Dataset

The LIAR Dataset is a publicly available benchmark for misinformation detection, sourced from PolitiFact.com and hosted on HuggingFace. It contains 12836 short political statements, each labelled by PolitiFact editors. The original dataset uses six labels: true,

mostly-true, half-true, barely-true, false and pants-fire. To simplify our task into a bit of a simpler task we mapped the labels as follows: true and mostly-true become “true”, half-true and barely-true become “mixed”, and false and pants-fire become “false”. This just simplifies the original 6 labels into 3 while keeping the meaning intact.

3.2 FakeNewsNet (PolitiFact)

FakeNewsNet is a dataset collected for fake news research. It contains articles from both PolitiFact and GossipCop. We chose to use only the PolitiFact portion of 1056 samples because we wanted to maintain domain consistency with the LIAR dataset which also draws from PolitiFact. Mixing in celebrity gossip headlines could confuse the models by introducing very different writing styles and vocabulary. Each article is labelled as either real or fake based on PolitiFact’s fact checking verdicts. We mapped the real label to our “true” label, and the fake label to our “false” label. Since this dataset is binary it doesn't contribute to the “mixed” label at all.

3.3 Combined Dataset

We combined both datasets into a single corpus of 13892 samples after dropping rows with mixed text. The final label distribution is as follows: true (5153 samples, 37.1%), mixed (4746 samples, 34.2%) and false (3993 samples, 28.7%). The classes are reasonably balanced but false is a little underrepresented. To address this we used balanced class weights in our baseline model and stratified splits across all experiments to ensure consistent class ratios in the training, validation and test sets. The LIAR dataset contributes the majority of the data while FakeNewsNet add some additional real and fake examples from the same political fact checking domain.

3.4 WordNet Integration

We integrated wordnet to improve langchains fact checking capabilities. There is two parts the first part is manual swapping because during testing it was missing some basic word swaps that could expand the searching. Then after manual swap, it then uses wordnet to find other words to swap. Afterwards it then searches all 3 queries and concatenates the results together to be analysed. This was changed based on noise it caused and increased the already very long query time by even more.

4. Modelling

4.1 Baseline: TF-IDF + Logistic Regression

The baseline model uses TF-IDF (term Frequency-Inverse Document Frequency) vectorisation paired with Logistic Regression. The TF-IDF vectoriser is configured with a vocabulary cap of 10000 features using both unigrams and bigrams with English stopwords

removed. This captures not just individual word importance but also common two word phrases that might signal misinformation like “breaking news” or “sources say”)

The Logistic Regression classifier uses balanced class weights to account for the slight imbalance across our three classes. We chose this as our baseline because it is a well established and easily interpretable approach for text classification. It also requires no pre-trained language model and provides a clear performance floor which we can measure against later when looking at the improvements the models give.

4.2 Pre-trained BERT

To provide a comparison point for fine tuning, we evaluated a pre-trained BERT model (bert-base-uncased) without any task specific training. The model was loaded with a randomly initialised classification head on top of the pre-trained transformer layers.

Since the classification head has not been trained on our data, this model makes basically random predictions across the three classes (~33% accuracy). The point of including this evaluation method is to serve as a reference point to demonstrate that BERT models general language understanding alone is not sufficient for misinformation detection. This means that the model must be explicitly fine tuned on labelled examples in order to learn the patterns that distinguish true, mixed and false statements.

4.3 Fine-tuned BERT

We fine tuned bert-base-uncased on our combined dataset using a 60/20/20 stratified split for training, validation and testing. This ensures the class distribution remains consistent across all splits. Additionally the test set is held out entirely until the final evaluation.

The fine tuning setup uses the AdamW optimiser with a learning rate of 1e-5 and a linear warmup schedule over the first 10% of training steps. We train for up to 5 epochs with early stopping. If validation accuracy does not improve for 2 consecutive epochs the training halts and the best performing model weights are restored. Input sequences are tokenised with a maximum length of 128 tokens which is enough for the short headlines in our dataset.

4.4 Fine-tuned RoBERTa

We fine-tuned RoBERTa (roberta-base) using the same training configuration as BERT to allow a fair comparison. RoBERTa was chosen because it builds on BERT’s architecture with a few key improvements. It uses dynamic masking, removes the next-sentence prediction objective and is trained on a larger corpus with larger batch sizes. These changes have been shown to improve performance on a range of downstream NLP tasks making it a good choice for a comparison model.

It was trained as above with the fine tuned bert model.

4.5 LangChain Integration

To enhance our language model, we integrate LangChain's tooling to assist our language models, specifically the Search and LLM / Agent tools. The decision logic leverages Google's Serper API to retrieve up-to-date external evidence on the inputted headline. Though, this data needs to be evaluated and interpreted into true, false, or mixed, and ideally a 'confidence level' that can be used to adjust our language model's decisions. So, the data is synthesized using LangChain's agent tool, with the LLM of choice being Gemini's 2.5 flash model.

In order to achieve a structured / consistent output from Gemini, we use python's Pydantic module to generate a verdict (true, false, mixed), confidence score (score from 0 to 1) and reasoning. The confidence score is suggested to Gemini to be determined via the number of reputable sources it found, and if contradictory evidence is found against the headline. This transformation allows us to gather real-time information through web searches, and transform the subjective assessment into a verifiable, evidence-informed verdict.

5. Evaluation

5.1 Quantitative Results

Model	Accuracy	Precision	Recall	Macro F1
TF-IDF + Logistic Regression	0.4430	0.4414	0.4425	0.4415
Pre-trained BERT	0.3699	0.2005	0.3326	0.1825
Fine-tuned BERT	0.4933	0.5100	0.4821	0.4810
Fine-tuned RoBERTa	0.5833	0.5146	0.5005	0.5772
RoBERTa + LangChain	0.6000	0.5999	0.6000	0.5947
RoBERTa+LangChain+WordNet	0.6000	0.6054	0.6000	0.5964

Accuracy: measures the proportion of predictions the model got correct out of all predictions made.
Precision: measures how many of the model's positive predictions for each class were actually correct. High precision means less false positives.

Recall: measures how many of the actual instances of each class the model successfully identified. High recall means fewer missed cases.

Macro F1: this is the harmonic mean of precision and recall averaged equally across all three classes. It provides a single balanced score that accounts for both false positives and missed cases and treats each class equally regardless of size.

Confusion matrices:

We can find from these what types of mistakes the models are making specifically.

Fine-tuned RoBERTa

	Predicted false	Predicted mixed	Predicted true
Actual false	9	6	5
Actual mixed	2	14	4
Actual true	4	4	12

RoBERTa + LangChain

	Predicted false	Predicted mixed	Predicted true
Actual false	10	5	5
Actual mixed	2	14	4
Actual true	4	4	12

RoBERTa + LangChain + WordNet

	Predicted false	Predicted mixed	Predicted true
Actual false	10	5	5
Actual mixed	2	14	4
Actual true	4	4	12

Output example

```
============================================================
ROBERTA + LANGCHAIN + WORDNET
============================================================

Accuracy: 0.5000

Classification Report:
              precision    recall  f1-score   support
```

```
       false     0.4444    0.4000    0.4211         10
       mixed     0.5833    0.7000    0.6364         10
        true     0.4444    0.4000    0.4211         10

    accuracy                         0.5000         30
   macro avg     0.4907    0.5000    0.4928         30
weighted avg     0.4907    0.5000    0.4928         30

Confusion Matrix:
[[4 3 3]
 [1 7 2]
 [4 2 4]]

Overrides:
None
Override Impact:
Corrected by LangChain: 0
Made worse by LangChain: 0
Unchanged correct: 14
Unchanged wrong: 16

WordNet Overrides:
mixed -> false: 1
```

5.2 Qualitative Error Analysis

Example
2. Headline: President Obama has the worst ratings of any president at the end of his first year.
Gold Label: true
ROBERTA Prediction: true (confidence: 0.4258)
LangChain Prediction: false (evidence verdict: false, confidence: 1.0000)
LangChain + WordNet Prediction: true (evidence verdict: false, confidence: 0.9000)

Quantitative analysis on the data sets show that the data set itself is outdated where the truth has changed with time. The langchain uses more recent knowledge to check if headlines are true or not, whilst the data bases are built on an older knowledge set when some headlines could have been true.

However somehow by expanding searches with wordnet determined that despite it believing it was false still gave it true, this was likely caused by the fact that it found a search to suggest that at some point in time it was true. Meaning temporal truth is an aspect we must make a decision on.

Another common example within the confusion matrices show high level of confusion for false results. The false results are much harder for the all 3 models to determine.

The langchain also doesn't make many changes to the results, as our original combination method which provided higher value to langchain was getting terrible results, so we made it more conservative to only change things that it was sure of and that roberta was unsure of.

5.3 Model Comparison

Tf-idf + logistic regression acts as a baseline showing that with its 0.4430 accuracy its capable of capturing some level of patterns in the data based on language. However it has no understanding or context of whats right and wrong.

Pre-trained Bert accuracy goes down to 0.3699 accuracy, meaning, showing that it is ineffective for this task without specific training so it can better understand the task, with more specific context.

Fine tuned bert shows higher accuracy beating the base line and pre trained bert, however still not showing significant capability of determining truthfulness of headlines.

Fine tuned Roberta has the best results for non retrieval results showing that it has better architecture for understanding the context of the headlines. Reaching a relatively significant jump from 0.49 to 0.58.

The Langchain Roberta combination has little advancement to improve the results from the Roberta model. This is caused by two aspects, first is the conservative nature of the model to not override a confident roberta model, as well as the fact it was only using 60 pieces of testing data.

Wordnet langchain roberta model shows no major changes to the model, showing that in this current testing environment it has very few effects on the result of being able to search different versions of the sentence. Likely because the gemini architecture is already advanced and isn't affected by small synonym changes that drastically. Especially since we concatenate all results together to be analysed which would reduce the effect of the different information found.

6. Demo and Interfaces

6.1 Command Line Interface

Example use.

python cli_test.py --model roberta --text "The unemployment rate has dropped to its lowest point in 50 years"

We can also add the flag --use-wordnet to also test wordnet additions.

–model ‘roberta’ chooses a model based on name. After running that model on the example it then performs the CLI retrieval fact checking step using langchain. This sends a headline

to google search tool and asks the language model to evaluate the claim using evidence from its search results.

If the wordnet flag is enabled it expands the query results to allow for more searches to be done based on the different variant sentences wordnet produces. It then combines the two results together from the –model and the cli results to make its new prediction. If roberta or bert is unsure langchain will override its prediction. The CLI output will also try to provide its reasoning and sources.

We also have evaluation commands which are

python evaluate_langchain.py --model roberta —limit 30

python evaluate_langchain.py --model bert --input path/to/file.csv --output results.txt

These two commands allow you to test a different amount of test data and compare results they will give you the aspects and scores for each model vs lang chain and wordnet models. They will display the accuracy and metrics for the overall model as well as the individual choices.

The input and output tests allow for you to read from a file of headlines then create an output to display every headline and see what each model expected for each of those headlines as well as a confidence score.

python generate_eval_data.py --per-label 20 --seed 42

for quick evaluation testing to create a dataset from untrained data from the validation set.

6.2 Gradio Demo

We have built an interactive web demo using Gradio that allows users to test the system in their browser. Follow the instructions on the readme to run the demo. This will launch a local webpage where you can enter a headline into the text box and select a model from the drop down menu. The prediction and confidence scores will then be displayed on the webpage.

The webpage also includes an optional section for the RAG work with LangChain and wordnet. You can tick a checkbox and paste in your google API key and it will run the evidence based fact checking system implemented using LancChain and google search alongside the model prediction. This lets you see the output of both the fine tuned models and RAG systems side by side.

7. Conclusion

We successfully developed and evaluated models and techniques for news misinformation detection, with notable improvement in accuracy over the developed iterations. Moving from a baseline TF-IDF approach to using pretrained, fine-tuned transformer architectures saw improvement in the model's ability to validate news headlines. The integration of LangChain and Serper API had much potential to provide real-time fact checking to collectively work with our fine-tuned models to provide a more comprehensive verification suite. The layered approach was effective in improving the ability of the models as shown in the results, though the use of higher-powered LLM's we may have found even more success. Ultimately, this project demonstrates the levels of effectiveness and ability to verify sources of news via NLP and external tooling to uphold the integrity of the digital news ecosystem.

References

LangChain Python documentation:
https://docs.langchain.com/oss/python/langchain/overview

WordNet Python documentation: https://docs.langchain.com/oss/python/langchain/overview